



# Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

|                  |                                         |
|------------------|-----------------------------------------|
| Student Name     | Om Mule                                 |
| PRN No           | 202401040325                            |
| Roll No          | 147                                     |
| Program          | Computer Engineering                    |
| Year             | Third Year                              |
| Division & Batch | B – B3                                  |
| Subject          | Operating Systems Laboratory (2304311L) |
| Assignment No    | Three                                   |

Problem Statement:

**Implement a simple process scheduler using CPU Scheduling Algorithms such as FCFS (First-Come, First-Served), SJF (Shortest Job First), and RR (Round Robin)**

**Code:**

```
#include<stdio.h>
#include<string.h>

int main()
{
    int choice;
    int n;
    int burst[20];
    int wait[20];
    int turn[20];
    int process[20];
    int remaining[20];
    float avg_wait;
    float avg_turn;

    do
    {
        printf("\n=====");
        printf("\n    CPU SCHEDULING MENU");
        printf("\n=====");
        printf("\n1. FCFS");
        printf("\n2. SJF");
        printf("\n3. Round Robin");
        printf("\n4. Exit");
        printf("\n=====");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        if (choice == 4)
```

```

    {
        printf("\nProgram terminated.\n");
        break;
    }

    printf("\nEnter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time for each process:\n");
    for (int i = 0; i < n; i++)
    {
        process[i] = i + 1;
        printf("P%d: ", i + 1);
        scanf("%d", &burst[i]);
    }
    //-----//FCFS
    if (choice == 1){
        avg_wait = 0;
        avg_turn = 0;
        wait[0] = 0;

        for (int i = 1; i < n; i++)
        {
            wait[i] = wait[i - 1] + burst[i - 1];
        }

        for (int i = 0; i < n; i++)
        {
            turn[i] = wait[i] + burst[i];
            avg_wait = avg_wait + wait[i];
            avg_turn = avg_turn + turn[i];
        }
        printf("\nGantt Chart:\n");

        for (int i = 0; i < n; i++)
        {
            printf("| P%d ", process[i]);
        }

        printf("\n0");
        int time = 0;

        for (int i = 0; i < n; i++)
        {
            time = time + burst[i];
            printf("----%d", time);
        }

        printf("\n");
        printf("\nProcess\tBT\tWT\tTAT\n");

        for (int i = 0; i < n; i++)
        {
            printf("P%d\t%d\t%d\t%d\n", process[i], burst[i], wait[i], turn[i]);
        }

        printf("\nAverage Waiting Time: %.2f", avg_wait / n);
        printf("\nAverage Turnaround Time: %.2f\n", avg_turn / n);
    }
    //-----//SJF
    if (choice == 2){
        for (int i = 0; i < n - 1; i++)
        {
            for (int j = i + 1; j < n; j++)
            {

```

```

        if (burst[i] > burst[j])
        {
            int temp;
            temp = burst[i];
            burst[i] = burst[j];
            burst[j] = temp;
            temp = process[i];
            process[i] = process[j];
            process[j] = temp;
        }
    }
}

avg_wait = 0;
avg_turn = 0;
wait[0] = 0;

for (int i = 1; i < n; i++)
{
    wait[i] = wait[i - 1] + burst[i - 1];
}

for (int i = 0; i < n; i++)
{
    turn[i] = wait[i] + burst[i];
    avg_wait = avg_wait + wait[i];
    avg_turn = avg_turn + turn[i];
}

printf("\nGantt Chart:\n");

for (int i = 0; i < n; i++)
{
    printf("| P%d ", process[i]);
}

printf("\n0");
int time = 0;

for (int i = 0; i < n; i++)
{
    time = time + burst[i];
    printf("    %d", time);
}

printf("\n");
printf("\nProcess\tBT\tWT\tTAT\n");

for (int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\n", process[i], burst[i], wait[i], turn[i]);
}

printf("\nAverage Waiting Time: %.2f", avg_wait / n);
printf("\nAverage Turnaround Time: %.2f", avg_turn / n);
}

//-----//Round Robin
if (choice == 3){
    int quantum;
    int time = 0;
    int completed = 0;
    avg_wait = 0;

```

```

    avg_turn = 0;

    printf("\nEnter Time Quantum: ");
    scanf("%d", &quantum);

    for (int i = 0; i < n; i++)
    {
        remaining[i] = burst[i];
        wait[i] = 0;
    }

    printf("\nGantt Chart:\n");

    while (completed < n)
    {
        for (int i = 0; i < n; i++)
        {
            if (remaining[i] > 0)
            {
                printf("| P%d ", process[i]);

                if (remaining[i] > quantum)
                {
                    time = time + quantum;
                    remaining[i] = remaining[i] - quantum;
                }
                else
                {
                    time = time + remaining[i];
                    remaining[i] = 0;
                    turn[i] = time;
                    wait[i] = turn[i] - burst[i];
                    completed++;
                }
            }
        }
    }
    printf("\n");
    printf("\nProcess\tBT\tWT\tTAT\n");

    for (int i = 0; i < n; i++)
    {
        avg_wait = avg_wait + wait[i];
        avg_turn = avg_turn + turn[i];
        printf("P%d\t%d\t%d\t%d\n", process[i], burst[i], wait[i], turn[i]);
    }

    printf("\nAverage Waiting Time: %.2f", avg_wait / n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_turn / n);
}
//-----//exit
else{
    printf("\nInvalid choice! Please try again.\n");
}
}while(choice != 4);
return 0;

```

## Output of the program:

```
omion@OM:~$ vim cpu_scheduling.c
omion@OM:~$ gcc cpu_scheduling.c -o cpu_scheduling.exe
omion@OM:~$ ./cpu_scheduling.exe

=====
CPU SCHEDULING MENU
=====
1. FCFS
2. SJF
3. Round Robin
4. Exit
=====
Enter your choice: 2

Enter number of processes: 3
Enter burst time for each process:
P1: 10
P2: 6
P3: 9

Gantt Chart:
| P2 | P3 | P1 |
0   6   15   25

Process BT    WT    TAT
P2      6      0      6
P3      9      6     15
P1     10     15     25

Average Waiting Time: 7.00
Average Turnaround Time: 15.33
```

```
=====
CPU SCHEDULING MENU
=====
1. FCFS
2. SJF
3. Round Robin
4. Exit
=====
Enter your choice: 1

Enter number of processes: 5
Enter burst time for each process:
P1: 2
P2: 4
P3: 6
P4: 7
P5: 8

Gantt Chart:
| P1 | P2 | P3 | P4 | P5 |
0---2---6---12---19---27

Process BT    WT    TAT
P1      2      0      2
P2      4      2      6
P3      6      6     12
P4      7     12     19
P5      8     19     27

Average Waiting Time: 7.80
Average Turnaround Time: 13.20
```

## Note book:

1/2/2019

# Unit III CPU Scheduling

DATE / /

Page No. /

Operating System in Spring

## (A) Intro:

- In single processor system only one process can run at a time.
- Process must wait until the CPU is free and can be rescheduled.
- Multiprogramming: The objective of multiprogramming is to maximize CPU utilization. In it multiple programs are kept on memory at a time like which one is waiting other is completing its I/O operations and so on hence CPU don't stays idled.

- Process execution consists of a cycle of CPU execution and I/O wait. i.e. CPU burst followed by I/O burst.
- CPU & I/O burst cycle: It is the amount of time required by CPU to finish a process or can be said the amount of time required by the process to finish.
- We can't estimate time taken by process before running it.
- I/O: While the process is running state, it may ask for I/O process goes to the block or wait state.

## (B) CPU Scheduler's

- Short-term Scheduler's select from among processes in ready queue and allocates CPU to one of them.

Queue may be ordered in various ways.

- CPU scheduling decisions may take place when a process
  1. Switching from running to waiting (I/O request generally)
  2. Switching from running to ready state (interrupt)
  3. Switching from waiting to ready (completion)
  4. Termination.

- Schedulers under 1 and 4 are preemptive all others are non-preemptive.

- Pre-emptive scheduling incurs a cost associated with access to shared data.

DATE / /

Page No. /

- Pre-emptive also effects design of OS kernel.
- Pre-emptive interrupts occur during critical OS activities.

## (B) Dispatcher's

- Another components involved in the CPU-scheduling function is the dispatcher.

- Dispatcher module give control of the CPU to the process selected by short term scheduler. It involves:
  - Switching context
  - Switching to user mode
  - Jumping to proper location

- The dispatcher set should be as fast as possible since it is involved during every process switch.
- Dispatcher latency: time taken for stopping and running other process by the dispatcher.

## (B) Scheduling Criteria's

- Different CPU-scheduling algorithms have different properties.
- Many criteria have been suggested for comparing CPU-scheduling algorithms.

- ↑ max CPU utilization: Number of processes the CPU will (100%)
- ↑ max Throughput: Number of process that complete per execution per unit.
- ↓ min Turnaround time: amount of time to execute a particular process.
- ↓ min Waiting time: end of time process has been waiting in ready queue.
- ↓ min Response time: end of time from when request was submitted until first response is produced, not output.

- Arrival time: time at which the process arrives in system for CPU.

## (C) Scheduling Algorithms

- CPU scheduling deal with problem of deciding which of process in ready queue is to be allocated CPU.

- There are many different CPU scheduling algorithms:
- First Come, First served (FCFS) scheduling - Non-preemptive
  - Shortest Job First (SJF) scheduling - Non-preemptive
    - ↳ shortest remaining time first scheduling - Preemptive
  - Priority scheduling - Non-preemptive & Preemptive
  - Round-Robin kind scheduling - Preemptive

Gantt chart:

|                |                |                |                |                |                |                |
|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 0-1            | 1-2            | 2-3            | 3-5            | 5-8            | 8-14           | 14-23          |
| P <sub>4</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>4</sub> | P <sub>3</sub> | P <sub>2</sub> | P <sub>1</sub> |
| 0              | 1              | 2              | 3              | 5              | 8              | 14             |

↓ 23

Average waiting time:  $\frac{13 + 6 + 2 + 0}{4} = \frac{21}{4} = 5.25 \text{ units}$

Average turnaround time =  $\frac{23 + 13 + 6 + 2}{4} = \frac{44}{4} = 11 \text{ units}$

|                | <u>WT</u> | <u>TAT</u> | 4    | 4           |
|----------------|-----------|------------|------|-------------|
| P <sub>1</sub> | 13        | 23         | 14-1 | TAT<br>23-0 |
| P <sub>2</sub> | 6         | 13         | 8-2  | 14-1        |
| P <sub>3</sub> | 2         | 6          | 5-3  | 8-2         |
| P <sub>4</sub> | 0         | 2          | 3-3  | 5-3         |

LAB  
17/Sept/26

OS Lab

Arrival time (AT):

The time at which process enters the ready queue.

Burst time (BT)

amount of time CPU requires to complete a process

Completion time (CT)

Time at which process completely finishes.  $CT = TAT + AT$

Turnaround time (TAT)

Total time a process spend/spends in the system.

$$TAT = CT - AT$$

$$TAT = WT + BT$$

Waiting Time (WT)

Time process spends waiting for CPU.

$$WT = TAT - BT$$

① FCFS (first come, first serve)

the process which comes first (in a order) is completed in that particular order, which arrives first completes first  
non-preemptive, means once a process gets CPU, it keeps the CPU until BT is completed.

$P1 \rightarrow P2 \rightarrow P3$

Gantt chart:

0      5      8      10

| P1 | P2 | P3 |

| Process | AT | BT |
|---------|----|----|
| P1      | 0  | 5  |
| P2      | 1  | 3  |
| P3      | 2  | 2  |

P1 runs first (5sec) then P2 then P3  
(3sec) (2sec)

after execution of P2 P1, P2 shall run while P3 is in arriving.

advantages: easy to implement and understand

Easy to maintain using a queue

fair in terms of arrival time / order.

No process is deliberately skipped or unfairly completed.

dis: If longest or big process arrives first followed by several

Small processes then short process needed to wait for longer time.

## ② SJF (Shortest Job First) :-

OS runs the processes which is shortest burst time first and then subsequently.

It's a non-preemptive

P2 → P3 → P4 → P1

Gantt chart:

|    | P1 | P2 | P3 | P4 |
|----|----|----|----|----|
| 0  |    |    |    |    |
| 1  |    |    |    |    |
| 5  |    |    |    |    |
| 10 |    |    |    |    |
| 14 |    |    |    |    |
| 18 |    |    |    |    |
| 22 |    |    |    |    |
| 26 |    |    |    |    |
| 30 |    |    |    |    |

SJF reduce average waiting time, SJF gives minimum avg waiting time among other scheduling algorithms.

advantages: Provides less/low waiting time

Short process finishes quickly.

Efficient when CPU burst time can be estimated

dis: starvation, consider like a long process P1 arrived but

is interrupting continuously new process with lower

burst time keeps arriving hence P1 shall not get CPU until

and still is in ready state.

## ③ RR (Round-Robin) :-

It is a time-sharing and interactive systems.

It's a Preemptive algorithm,

like consider every available process should receive a fixed

amount of time on CPU for execution (Time Quantum). after it

completes next process shall get CPU and new process is either

terminated (if completed) else goes in waiting.

P1 = 5 ms

P2 = 3 ms

Gantt chart :-

|     | P1 | P2 | P3 | P4 | P1 | P2 | P4 | P1 | P4 | P4 | P2 |
|-----|----|----|----|----|----|----|----|----|----|----|----|
| 0   |    |    |    |    |    |    |    |    |    |    |    |
| 2   |    |    |    |    |    |    |    |    |    |    |    |
| 4   |    |    |    |    |    |    |    |    |    |    |    |
| 6   |    |    |    |    |    |    |    |    |    |    |    |
| 8   |    |    |    |    |    |    |    |    |    |    |    |
| 10  |    |    |    |    |    |    |    |    |    |    |    |
| 12  |    |    |    |    |    |    |    |    |    |    |    |
| 14  |    |    |    |    |    |    |    |    |    |    |    |
| 16  |    |    |    |    |    |    |    |    |    |    |    |
| 18  |    |    |    |    |    |    |    |    |    |    |    |
| 20  |    |    |    |    |    |    |    |    |    |    |    |
| 22  |    |    |    |    |    |    |    |    |    |    |    |
| 24  |    |    |    |    |    |    |    |    |    |    |    |
| 26  |    |    |    |    |    |    |    |    |    |    |    |
| 28  |    |    |    |    |    |    |    |    |    |    |    |
| 30  |    |    |    |    |    |    |    |    |    |    |    |
| 32  |    |    |    |    |    |    |    |    |    |    |    |
| 34  |    |    |    |    |    |    |    |    |    |    |    |
| 36  |    |    |    |    |    |    |    |    |    |    |    |
| 38  |    |    |    |    |    |    |    |    |    |    |    |
| 40  |    |    |    |    |    |    |    |    |    |    |    |
| 42  |    |    |    |    |    |    |    |    |    |    |    |
| 44  |    |    |    |    |    |    |    |    |    |    |    |
| 46  |    |    |    |    |    |    |    |    |    |    |    |
| 48  |    |    |    |    |    |    |    |    |    |    |    |
| 50  |    |    |    |    |    |    |    |    |    |    |    |
| 52  |    |    |    |    |    |    |    |    |    |    |    |
| 54  |    |    |    |    |    |    |    |    |    |    |    |
| 56  |    |    |    |    |    |    |    |    |    |    |    |
| 58  |    |    |    |    |    |    |    |    |    |    |    |
| 60  |    |    |    |    |    |    |    |    |    |    |    |
| 62  |    |    |    |    |    |    |    |    |    |    |    |
| 64  |    |    |    |    |    |    |    |    |    |    |    |
| 66  |    |    |    |    |    |    |    |    |    |    |    |
| 68  |    |    |    |    |    |    |    |    |    |    |    |
| 70  |    |    |    |    |    |    |    |    |    |    |    |
| 72  |    |    |    |    |    |    |    |    |    |    |    |
| 74  |    |    |    |    |    |    |    |    |    |    |    |
| 76  |    |    |    |    |    |    |    |    |    |    |    |
| 78  |    |    |    |    |    |    |    |    |    |    |    |
| 80  |    |    |    |    |    |    |    |    |    |    |    |
| 82  |    |    |    |    |    |    |    |    |    |    |    |
| 84  |    |    |    |    |    |    |    |    |    |    |    |
| 86  |    |    |    |    |    |    |    |    |    |    |    |
| 88  |    |    |    |    |    |    |    |    |    |    |    |
| 90  |    |    |    |    |    |    |    |    |    |    |    |
| 92  |    |    |    |    |    |    |    |    |    |    |    |
| 94  |    |    |    |    |    |    |    |    |    |    |    |
| 96  |    |    |    |    |    |    |    |    |    |    |    |
| 98  |    |    |    |    |    |    |    |    |    |    |    |
| 100 |    |    |    |    |    |    |    |    |    |    |    |

advantages: Provides fairness for process execution

Suitable for interactive and time sharing systems

Prevents one process for holding CPU indefinitely.

Provide reasonable response time when TQ is small.

dis: performance depends on time quantum heavily.

Context switching dependent heavily.